

ASSIGNMENT - 3

Loops and Iterations — while, for, do-while, Infinite Loop, break, continue & Nested Loops — IIITA Campus Life Problems

Course	Problem Solving with Programming (C)
Program	B.TECH — 2026 Batch
Section	IT-SEC A
Assignment No.	3
Subject Area	Loops and Iterations: while, for, do-while, infinite loop, break, continue, nested loops
Theme	Real IIITA Campus Locations & Situations
Total Programs	10 (each with code, output and line-by-line explanation)

INSTRUCTIONS

Read the full PDF very carefully. Part A explains what a loop is and every loop concept of this assignment one by one, with a real IIITA example on the left and the matching C code on the right. Part B contains the 10 practice programs based on those concepts.

Today's Class Task

1. Run all **10 programs** given in Part B on your laptop or college system through VS Code.
2. Take a screenshot of the **code and output** for every question.
3. Paste all 10 screenshots into **one Google Doc**, in order from Question 1 to Question 10.
4. Name that Google Doc using your own **Enrollment Number**.
5. Submit it on **Google Classroom** by clicking the '**Turn In**' button.

Note — the comment lines inside the programs

Lines written as `// ...` or `/* ... */` inside a program are **comments**. The compiler ignores them completely, so they never change the output. They are written for the programmer: to keep the logic clear, and to remember why a line was written. Keep such comments in your own submitted programs as well.

Table of Contents

Part A — Concepts (with real IIITA examples)

1. What is a Loop? What is an Iteration? — Jogging Rounds at the SAC
2. The Four Parts of Every Loop — the Main Gate Entry Counter, and choosing $\leq$ or $<$
3. The while Loop — Rounds at the SAC Track
4. The for Loop — Issue Slips at the Library Counter
5. The do-while Loop — PC Check at CC3
6. The Infinite Loop — the Main Gate Barrier that Never Stops
7. The break Statement — in while, do-while, for and switch
8. The continue Statement — in for, while and do-while
9. Nested Loops — Rows and Seats at the IIITA Pavilion

Part B — Practice Programming Questions (10 Programs)

- Q1. SAC Jogging Track — Counting Rounds (while)
 - Q2. IIITA to Civil Lines — Counting the Kilometre Markers (for)
 - Q3. IIITA Library — Stamping the Due Date on Books (do-while)
 - Q4. Prayagraj Junction — Announcements Until the Train Leaves (infinite loop + break)
 - Q5. Room 5006 — Looking for an Empty Seat (break inside for)
 - Q6. IIITA Main Gate — the Night Entry Register (continue inside for)
 - Q7. IIITA to Sangam — the Bus that Skips an Empty Stop (continue inside while)
 - Q8. Campus Trip Desk — Sangam, Civil Lines, Junction (break inside switch)
 - Q9. IIITA Pavilion — Rows and Seats (nested for)
 - Q10. Weekly Timetable — Lab 5042 and Theory 5006 (nested loop with continue and break)
- Quick Revision** — Concept, Real IIITA Memory, Code Shape

PART A — CONCEPTS

In Assignment 2, the program learned to **decide** — it could pick one path out of two or more. But every program you have written so far still did each job **only once**. If a message had to be printed six times, you had to write six `printf` lines.

In Assignment 3 the program learns to **repeat**. One small block of code, written once, can run five times, fifty times or five thousand times. This is what **loops** are for, and each single run of that block is called one **iteration**.

Everything in this assignment is built on that one idea. Concept 1 explains it with something you have all seen at the SAC, before a single line of code appears.

CONCEPT 1

1. What is a Loop? What is an Iteration?

REAL IIITA SITUATION — THE JOGGING TRACK AT THE SAC

In the evening you go to the **Student Activity Centre (SAC)** and decide: "**I will jog 5 rounds of the track.**"

What do you actually do? You run the *same* track again and again. The track does not change. Only one thing changes — the **round number** in your head: 1, 2, 3, 4, 5. Once the fifth round is finished, you stop and walk back to the hostel.

That whole activity is a **LOOP**: one fixed job repeated again and again.

One single round of the track is one **ITERATION**. Five rounds means the loop ran **five iterations**.

THE SAME IDEA IN C

Without a loop, five rounds would have to be written five times:

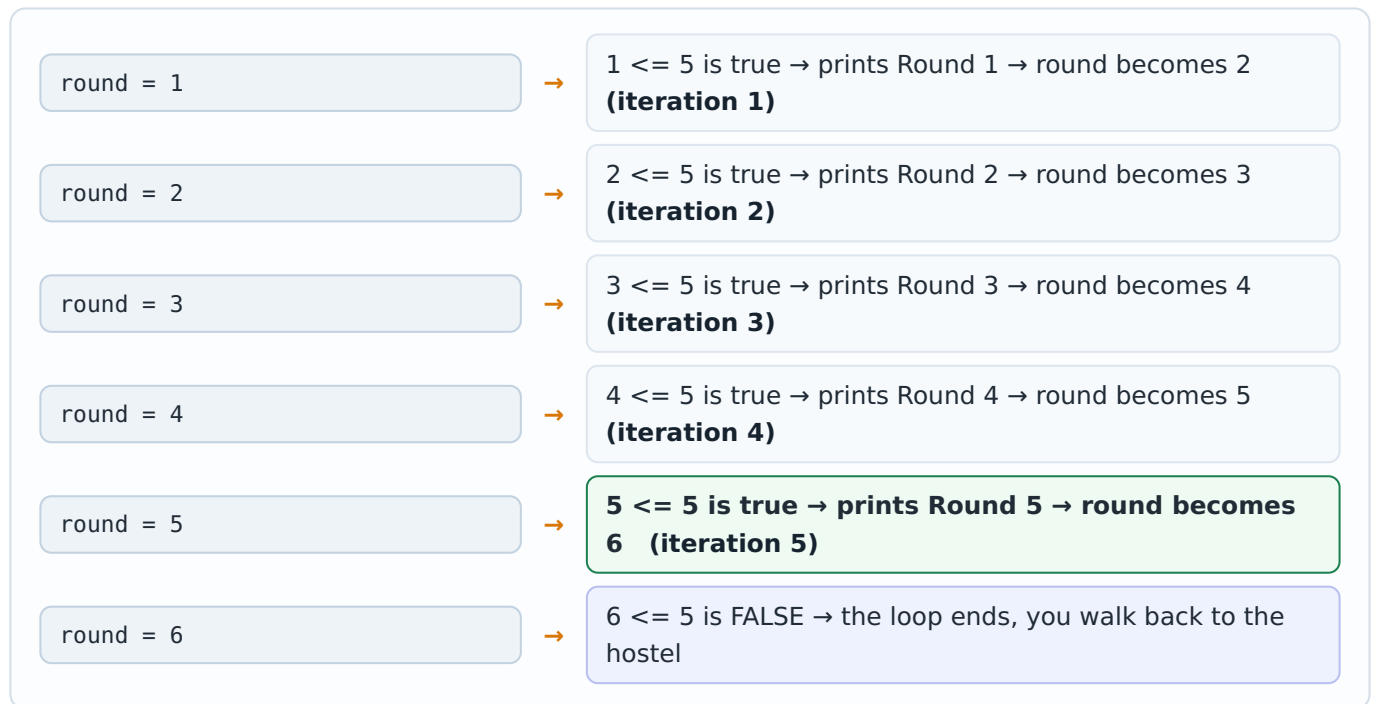
```
printf("Round 1\n");
printf("Round 2\n");
printf("Round 3\n");
printf("Round 4\n");
printf("Round 5\n");
```

With a loop, the same work is written **once** and repeated by the computer:

```
int round = 1;

while (round <= 5) {
    printf("Round %d\n", round);
    round++;
}
```

Step by step — what the computer actually does for 5 rounds



AT THE SAC TRACK	→	IN THE PROGRAM
The whole activity of jogging rounds	→	the LOOP
One single round of the track	→	one ITERATION
The round number you keep in your head	→	int round = 1;
"Is there still a round left to run?"	→	while (round <= 5)
Running one more round	→	round++;

Remember these two words

Loop = the repetition itself, the job that runs again and again.

Iteration = one single run of that job. Five rounds at the SAC = one loop, five iterations.

2. The Four Parts of Every Loop

Every loop you will ever write has the same four parts. Once you can spot these four, all three loop types (while, for, do-while) become easy, because they only arrange these same four parts differently.

REAL IIITA SITUATION — THE ENTRY COUNTER AT THE MAIN GATE

The guard at the **IIITA Main Gate** is counting students entering in the morning. His work has exactly four parts:

- **Start:** he begins his count at 1
- **Check:** "is my count still 5 or less?" — asked before every entry
- **Work:** he notes the student in the register
- **Update:** he moves his count one forward

Remove the **update** and his count stays at 1 for ever — he would keep counting the same student again and again and never finish. That mistake has a name in C too, and it is Concept 6.

THE SAME FOUR PARTS IN C

```
int student = 1;           // 1 INITIALISATION
while (student <= 5) {     // 2 CONDITION
    printf("Student %d\n",
           student);       // 3 BODY
    student++;             // 4 UPDATE
}
```

PART	WHAT IT DOES	AT THE MAIN GATE	HOW OFTEN IT RUNS
Initialisation	gives the counter its starting value	the guard starts his count at 1	only once , before the loop
Condition	decides whether to repeat once more	"is my count still 5 or less?"	before every iteration
Body	the actual work being repeated	noting the student in the register	once in every iteration
Update	moves the counter forward	the count goes one ahead	once in every iteration

Choosing the condition — when to write `<=` and when to write `<`

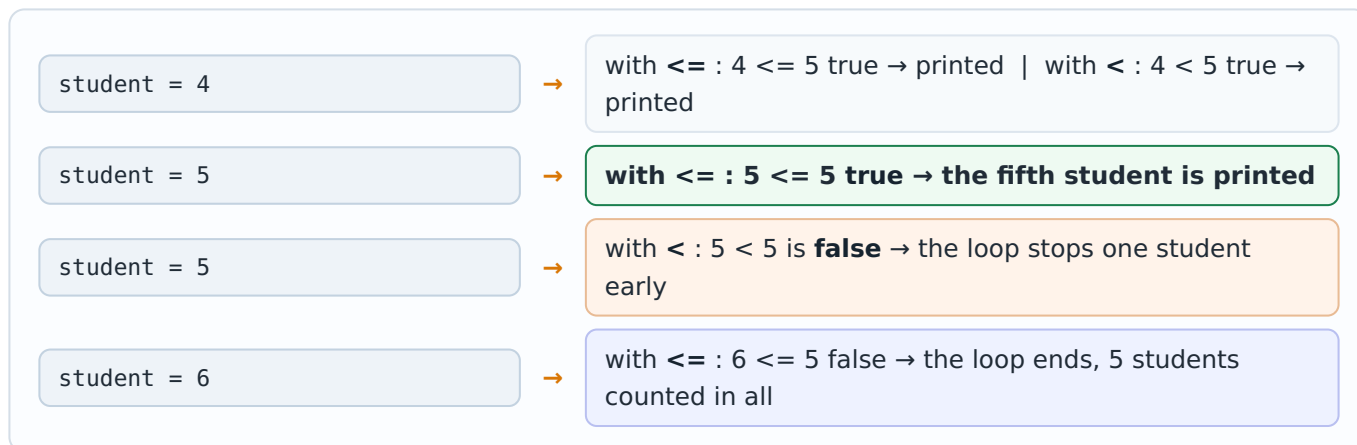
This one symbol decides **how many times** the loop runs. The correct choice depends on **where the counter starts** and **where the update is written**.

COUNTER STARTS AT 1 → USE `<=`

```
int student = 1;
while (student <= 5) {
    printf("Student %d\n", student);
    student++;
}
// prints Student 1 to Student 5
// that is 5 students
```

SAME LOOP WITH `<` → ONE STUDENT IS LOST

```
int student = 1;
while (student < 5) {
    printf("Student %d\n", student);
    student++;
}
// prints Student 1 to Student 4 only
// the fifth student is never counted
```



THE COUNTER STARTS AT...	THE UPDATE IS WRITTEN...	WRITE	WHY
1	at the bottom of the body, or in the for line	<code><=</code> last	the last number must still get a turn, so it has to make the condition true
0	at the top of the body (the counter moves up before the work)	<code><</code> total	the counter is already increased before the work, so <code><</code> already allows the last turn

The easy way to check it, every time

Put the last number into the condition yourself and see whether it comes out true:

"Should student 5 be counted?" → yes → so `5 <= 5` must be true → write `<=`.

If you had written `student < 5`, then `5 < 5` is false and the fifth student is dropped.

You will see both forms in this assignment, and both are correct in their own place: Question 1 counts from 1 and uses `round <= 8`; Question 7 counts from 0 with the update at the top of the body and uses `stop < 6`. Both run their loop the full number of times.

The most common mistake of the whole assignment

If you forget the **update**, the condition can never become false and the loop never ends. Whenever a program of yours keeps printing endlessly, look for the missing `round++` or `student++` first.

3. The while Loop

REAL IITA SITUATION — ROUNDS AT THE SAC TRACK

At the SAC you keep jogging **while** this sentence stays true: *"I have not finished 5 rounds yet."*

Notice **when** you check it. You check it **before** starting each round, standing at the starting line. If you have already done 5 rounds, you do not start a sixth one at all.

So if the target were **zero** rounds, you would not jog at all — the check comes first, so the work may never happen even once.

WHILE SYNTAX

```
while (condition) {
    // body: runs while the
    // condition stays TRUE
}
```

Use while when you do **not** know in advance how many times the work will repeat — you only know the condition that keeps it going.

The token counter at **CC1** works the same way: tokens keep being handed out *while* students are still standing in the queue. Nobody knows the exact number in advance.

START — you reach the SAC track, round = 1

CHECK FIRST: is round ≤ 5 ?

↓ one of the two ↓

TRUE — jog one more round

- ↓ body runs: "Round %d" is printed
- ↓ update runs: round++
- ↓ go back up and check again

FALSE — the rounds are over

- ↓ the body is skipped completely
- ↓ the loop ends
- ✓ you walk back to the hostel

END — the line after the loop runs → return 0;

Golden rule of while

The condition is tested **before** every iteration, including the very first one. So a while loop can run **zero** times if the condition is false at the start. This one sentence is the whole difference between while and do-while (Concept 5).

4. The for Loop

REAL IIITA SITUATION — ISSUE SLIPS AT THE LIBRARY COUNTER

At the **IIITA Library** counter, the librarian has to print **exactly 6 issue slips** for the six books on the trolley.

Here the number is **known before the work starts**. Nobody has to keep asking "should I print one more?" — it is simply slip 1, 2, 3, 4, 5, 6 and then stop.

For this kind of counting job, C gives a shorter loop where the start, the check and the update are all written together on one line, so nothing can be forgotten.

FOR SYNTAX

```
for (start; condition; update) {
    // body
}
```

```
int slip;

for (slip = 1; slip <= 6; slip++) {
    printf("Slip %d\n", slip);
}
```

The same four parts, only arranged differently

PART	IN A WHILE LOOP	IN A FOR LOOP
Initialisation	int slip = 1; (above the loop)	slip = 1 (first part of the for line)
Condition	while (slip <= 6)	slip <= 6 (middle part)
Update	slip++; (last line of the body)	slip++ (third part)
Body	inside { }	inside { }

Step by step — printing 6 slips

slip = 1



1 <= 6 true → prints Slip 1 → slip becomes 2

slip = 2 ... 5



the same four steps repeat, printing Slip 2, 3, 4 and 5

slip = 6



6 <= 6 true → prints Slip 6 → slip becomes 7

slip = 7



7 <= 6 is FALSE → the loop ends, all six slips are printed

while or for — which one to use?

for → when the number of repetitions is **known in advance** (6 slips, 3 rows of chairs, 10 PCs in a row).

while → when you only know the **condition** that keeps the work going, not the count. Both can do the same job; the for loop simply keeps the start, the check and the update together on one line so none of them gets forgotten.

5. The do-while Loop

REAL IIITA SITUATION — CHECKING A PC BEFORE THE LAB AT CC3

Before a lab at **CC3**, you sit at a PC and check whether it is working. You cannot know whether it works **until you have switched it on at least once**.

So the order is the opposite of the SAC track: here you **do the work first** and **ask the question afterwards** — "is there another PC left to check?"

Any job that must happen **at least once**, no matter what, is a do-while job. Even if there were no other PCs at all, this one PC has already been checked.

DO-WHILE SYNTAX

```
do {
    // body runs FIRST
} while (condition); // checked after
```

Note the **semicolon** after the closing bracket of `while (condition);` — it is needed only in a do-while, and forgetting it is a common error.

The one difference that matters — while vs do-while

LOOP	ORDER OF WORK	IF THE CONDITION IS FALSE RIGHT AT THE START
while	CHECK first, then do the work	the body runs zero times
do-while	DO the work first, then check	the body still runs once

WHILE — NOTHING IS PRINTED

```
int pc = 10;

while (pc <= 4) {
    printf("Checked\n");
    pc++;
}
// prints nothing at all
```

DO-WHILE — PRINTED ONCE

```
int pc = 10;

do {
    printf("Checked\n");
    pc++;
} while (pc <= 4);
// prints Checked once
```

AT CC3	→	IN THE PROGRAM
Switch on this PC and check it	→	<code>do { ... } // happens at least once</code>
"Is there another PC left to check?"	→	<code>} while (pc <= 4);</code>
Move to the next PC	→	<code>pc++; // inside the body</code>

6. The Infinite Loop

REAL IITA SITUATION — THE BARRIER AT THE MAIN GATE

The barrier at the **IIT Main Gate** works all day without stopping. It lifts, it lowers, it lifts again — nobody tells it "do this only 50 times". It simply keeps going **until the gate is closed at night**.

A loop like this, with no counting and no natural stopping point, is an **infinite loop**. It ends only when something inside it says "stop now" — the closing time.

An infinite loop can happen in two very different ways: **on purpose** (like the barrier) or **by mistake** (when the update is forgotten). The first is useful, the second is a bug.

ON PURPOSE — AND HOW IT IS STOPPED

```
while (1) {           // 1 is always true
    // the barrier keeps working
    if (closingTime) {
        break;        // the only way out
    }
}
```

while (1) and for (; ;) both mean "repeat for ever". Such a loop **must** contain a break, otherwise the program never finishes.

By mistake — the same loop with one line missing

WRONG — THE UPDATE IS MISSING

```
int student = 1;

while (student <= 5) {
    printf("Student %d\n", student);
    // student++ is missing
}
// student stays 1 for ever,
// so it prints Student 1 endlessly
```

CORRECT — THE UPDATE IS THERE

```
int student = 1;

while (student <= 5) {
    printf("Student %d\n", student);
    student++;        // the fix
}
// prints Student 1 to Student 5
// and then stops
```

If your program never stops

Press **Ctrl + C** in the terminal to stop it, then check three things in this order: is the **update** missing? is the update moving the counter in the **wrong direction**? is the **condition** one that can never become false? Nine out of ten endless programs are caused by the first of these three.

7. The break Statement

REAL IIITA SITUATION — THE SHUTTLE TO PRAYAGRAJ JUNCTION

A shuttle is picking up students at the Main Gate for **Prayagraj Railway Junction**. There are 10 students waiting, but the shuttle has only **6 seats**.

The driver takes student 1, 2, 3, 4, 5, 6 — and the sixth seat is now full. When student 7 steps up there is **no seat left**, so the boarding **stops there at once** and the bus drives off. Students 8, 9 and 10 are never even called.

He does not wait for the queue to finish naturally. He ends the repetition early, from inside. That is exactly what **break** does.

WHAT BREAK DOES

break stops the loop **immediately** and jumps to the first line written after the loop. The remaining iterations never happen.

```
// student = the place in the queue
for (student = 1; student <= 10; student++) {
    if (student > 6) {
        break;    // shuttle is full
    }
    printf("Student %d boards\n", student);
}
// control lands here after break
```

Step by step — 10 students in the queue, 6 seats in the shuttle

student = 1 ... 6



the condition student > 6 is false → the student boards and takes a seat

student = 7



7 > 6 is true — no seat left → break runs

break



the loop ends at once; the update student++ does not run either

student = 8, 9, 10



never reached — so students 7, 8, 9 and 10, four in all, are left at the gate

break works the same way in all four places

WHERE	WHAT BREAK DOES THERE	CAMPUS EXAMPLE
while	ends the while loop at once	counting entries at the gate stops when the register is full
do-while	ends the do-while at once, and the condition at the bottom is not even tested	the first book is stamped once even if no other book is waiting
for	ends the for loop at once; the update part does not run	boarding stops when the shuttle is full; the search stops when an empty seat is found (Question 5)
switch	ends the switch, so the cases written below do not run (this is the fall-through you saw in Assignment 2)	one trip is booked, the counter does not book the others too

One break, one loop

A break ends only the loop it is standing in. Inside a nested loop (Concept 9), a break in the inner loop ends the **inner** loop only — the outer loop keeps running with its next value.

8. The continue Statement

REAL IIITA SITUATION — ARRANGING THE CHAIRS AT THE AUDITORIUM

Before a programme at the **IIITA Auditorium**, ten chairs numbered 1 to 10 have to be arranged on the stage.

Chair **5** and chair **9** are **broken**. The worker does not stop the whole job and walk away — he simply **leaves those two chairs aside** and carries on with the next number.

That is **continue**: skip the rest of this one turn, and go straight to the next turn of the same loop. Eight chairs end up on the stage.

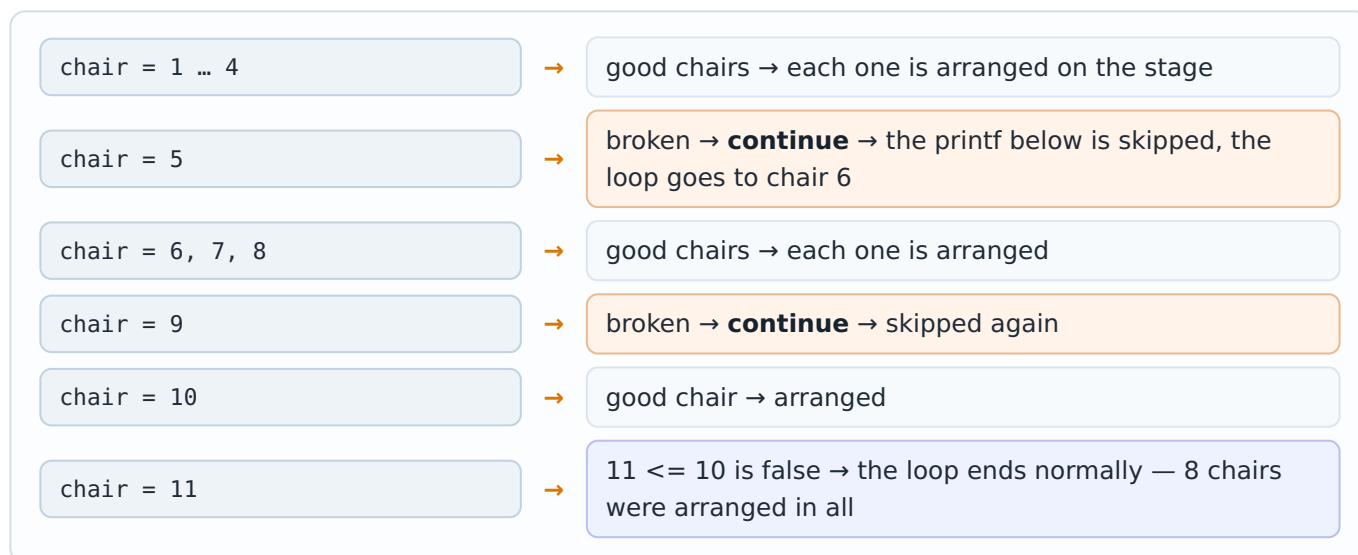
BREAK VS CONTINUE

break = leave the whole loop and never come back.

continue = leave only this one turn and come back for the next one.

```
for (chair = 1; chair <= 10; chair++) {
    if (chair == 5 || chair == 9) {
        continue; // broken, leave it
    }
    printf("Chair %d arranged\n", chair);
}
```

Step by step — going down the ten chairs



Where continue jumps to, in each loop

LOOP	CONTINUE JUMPS TO	CAREFUL ABOUT
for	the update part (i++), and then the condition	nothing — the counter still moves forward safely
while	straight back to the condition	the update is inside the body, so it must be done before the continue, or the loop never ends
do-while	down to the condition at the bottom	the same warning as while: update the counter before the continue

The continue trap in a while loop

In a for loop the update runs even after a continue, so nothing breaks. In a while loop it does not. If the update is written after the continue, the counter stops moving and the program runs for ever. In Question 7 the stop++ is deliberately written at the **top** of the body for exactly this reason.

9. Nested Loops

REAL IIITA SITUATION — ROWS AND SEATS AT THE PAVILION

At the **IIITA Pavilion**, chairs are being arranged for a match: **3 rows**, and **4 seats in every row**.

How do you number them? You go to row 1 and number its seats 1, 2, 3, 4. Only when that row is finished do you move to row 2 and number its four seats. Then row 3.

That is a **loop inside another loop**. The outer loop walks through the rows; for each single row, the inner loop runs completely, from seat 1 to seat 4.

Total chairs numbered = 3 rows × 4 seats = **12**, so the inner body runs 12 times in all.

NESTED LOOP IN C

```
for (row = 1; row <= 3; row++) {
    for (seat = 1; seat <= 4; seat++) {
        printf("R%d S%d ", row, seat);
    }
    printf("\n");    // row finished
}
```

The `printf("\n")` is in the **outer** body, so it runs once per row, not once per seat. That is what puts each row on its own line.

Step by step — the outer loop turns once, the inner loop turns four times

row = 1



inner loop runs fully: R1 S1, R1 S2, R1 S3, R1 S4 → then a new line

row = 2



inner loop starts again from seat 1: R2 S1, R2 S2, R2 S3, R2 S4 → new line

row = 3



inner loop runs fully once more: R3 S1, R3 S2, R3 S3, R3 S4 → new line

row = 4



4 <= 3 is false → the outer loop ends. Inner body ran 3 × 4 = 12 times

Two rules for nested loops

1. The inner loop restarts every time. Its counter goes back to its starting value at the beginning of each outer turn — seat starts again at 1 for every new row.

2. Use different counter names. The outer loop and the inner loop must not share the same variable, or the inner loop will destroy the outer loop's count. Here they are row and seat.

PART B — PRACTICE PROGRAMMING QUESTIONS

Each of the 10 programs below is built on a concept from Part A, and is given with its real IIITA situation, the complete program, a line-by-line explanation, and the exact output.

Which concept each question uses

QUESTION	CONCEPT USED
Q1 — SAC jogging rounds	the while loop (Concepts 1, 2 and 3)
Q2 — Kilometre markers to Civil Lines	the for loop (Concept 4)
Q3 — Stamping books at the Library	the do-while loop (Concept 5)
Q4 — Announcements at Prayagraj Junction	infinite loop stopped by break (Concepts 6 and 7)
Q5 — Looking for an empty seat in Room 5006	break inside a for loop (Concept 7)
Q6 — Main Gate night register	continue inside a for loop (Concept 8)
Q7 — Bus stops on the way to Sangam	continue inside a while loop (Concept 8)
Q8 — Campus trip desk	break inside a switch (Concept 7)
Q9 — Pavilion rows and seats	nested for loops (Concept 9)
Q10 — Weekly timetable	nested loops with continue and break (Concepts 7, 8 and 9)

Q1. SAC Jogging Track — Counting the Rounds

REAL SITUATION

Evening, 6:00 PM — the jogging track at the Student Activity Centre (SAC).

You decide to jog **8 rounds**. Before starting each round you ask yourself one question: *"is there still a round left to run?"*

As long as the answer is yes, you run one more round and add one to your count. Once the eighth round is finished, the answer becomes no and you walk back.

One round of the SAC track is **400 metres**, so at the end the program also works out how far you jogged.

CONCEPT USED

The **while** loop — the condition is checked **before** every round.

The four parts

start: round = 1

condition: round <= 8

body: print the round

update: round++

```
#include <stdio.h>

int main() {
    int round = 1;           // 1 INITIALISATION
    int distance;

    while (round <= 8) {     // 2 CONDITION
        printf("Round %d completed at the SAC track\n", round); // 3 BODY
        round++;             // 4 UPDATE
    }

    distance = (round - 1) * 400; // one round of the track is 400 metres
    printf("Jogging finished - %d rounds, %d metres in all", round - 1, distance);

    return 0;
}
```

AT THE SAC TRACK	→	IN THE CODE
You start counting from round 1	→	int round = 1;
"Is there still a round left to run?"	→	while (round <= 8)
Running one round of the track	→	printf("Round %d ...", round);
One more round is over	→	round++;
You stop and walk back	→	the line printed after the loop

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int round = 1;</code>	The counter, started at 1 because the first round is round number 1. It is written above the loop so that it is set only once.
<code>while (round <= 8)</code>	The condition, checked before every round. <= is used, not <, because round 8 must also run.
<code>printf(... , round);</code>	The body — the work being repeated. The value of round is different in every iteration, which is why the eight printed lines are not identical.
<code>round++;</code>	The update. Without this one line the count stays at 1, the condition stays true and the program prints Round 1 endlessly.
<code>distance = (round - 1) * 400;</code>	When the loop ends, round is 9, so round - 1 is 8 — the number of rounds actually finished. The program works out both the round count and the distance instead of having them typed into the message.
<code>printf("Jogging finished ...")</code>	Written outside the braces, so it runs only once, after the loop has ended.

OUTPUT

```
Round 1 completed at the SAC track
Round 2 completed at the SAC track
Round 3 completed at the SAC track
Round 4 completed at the SAC track
Round 5 completed at the SAC track
Round 6 completed at the SAC track
Round 7 completed at the SAC track
Round 8 completed at the SAC track
Jogging finished - 8 rounds, 3200 metres in all
```

Q2. IIITA to Civil Lines — Counting the Kilometre Markers

REAL SITUATION

Sunday evening — cycling from the IIITA main gate to Civil Lines.

The road is **6 kilometres** long, and there is a marker stone at every kilometre.

You know the number before you even start pedalling. It is simply marker 1, 2, 3, 4, 5, 6 — and then you have arrived. Nobody has to keep asking "is there one more?"

CONCEPT USED

The **for** loop — used when the number of repetitions is **known in advance**.

The start, the condition and the update all sit on one line:

```
for (km = 1; km <= 6; km++)
```

The counter starts at 1, so the condition uses `<=` — otherwise the sixth marker would never be counted.

```
#include <stdio.h>

int main() {
    int km;

    // start: km = 1   condition: km <= 6   update: km++
    for (km = 1; km <= 6; km++) {
        printf("Kilometre marker %d crossed\n", km);
    }

    printf("Civil Lines reached after %d kilometres", km - 1);

    return 0;
}
```

ON THE ROAD TO CIVIL LINES	→	IN THE CODE
The road is 6 kilometres long	→	<code>km <= 6</code>
Start counting from the first marker	→	<code>km = 1</code>
Crossing one marker stone	→	<code>printf("Kilometre marker %d crossed\n", km);</code>
Pedal on to the next marker	→	<code>km++</code>

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int km;</code>	Only declared here; its starting value is given inside the for line itself, which is the normal style for a for loop.
<code>for (km = 1; km <= 6; km++)</code>	All three parts on one line: start, condition, update. They are separated by semicolons , not commas.
<code>km <= 6</code>	The counter starts at 1, so <= is needed. With <code>km < 6</code> the loop would stop after marker 5 and you would never reach Civil Lines.
<code>{ printf(...); }</code>	The body. It runs six times, once for each value of km from 1 to 6.
<code>printf("... %d kilometres", km - 1);</code>	When the loop ends, km is 7, so km - 1 is 6 — the distance actually covered, worked out by the program.
<code>no ; after the for line</code>	Writing <code>for (...);</code> with a semicolon would give the loop an empty body, and the printf would run only once. This is a very common mistake.

OUTPUT

```
Kilometre marker 1 crossed
Kilometre marker 2 crossed
Kilometre marker 3 crossed
Kilometre marker 4 crossed
Kilometre marker 5 crossed
Kilometre marker 6 crossed
Civil Lines reached after 6 kilometres
```

Q3. IITA Library — Stamping the Due Date on Books

REAL SITUATION

Morning — the issue counter of the IITA Library.

4 books are lying on the counter, and the librarian has to stamp the due date on each one.

Notice the order: the librarian **stamps the first book**, and only after that does he look at the counter to see whether another book is waiting. The stamping happens before the question is asked.

The second half of the program shows the special power of do-while: even when the count has **already crossed** the limit, the body still runs **one time**.

CONCEPT USED

The **do-while** loop — the body runs **first**, and the condition is checked at the **bottom**.

So a do-while always runs **at least once**, while a plain while can run zero times.

Note the **semicolon** after while (condition); at the end.

```
#include <stdio.h>

int main() {
    int book = 1;
    int extra = 12;    // used in PART 2 below

    // PART 1 : four books are stamped one after another
    do {
        printf("Book %d stamped at the Library counter\n", book);
        book++;
    } while (book <= 4);

    // PART 2 : the count is already past the limit, yet the body still runs once
    do {
        printf("\nCount is already %d, but this line still runs one time\n", extra);
        extra++;
    } while (extra <= 4);

    return 0;
}
```

AT THE LIBRARY COUNTER	→	IN THE CODE
This book gets its due-date stamp	→	do { printf(...); book++; }
"Is another book waiting on the counter?"	→	} while (book <= 4);
Even with no book left, one stamp has already been made	→	PART 2: extra = 12, body still runs once

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>do { ... }</code>	The body. It is written above the condition, which is why it always runs before any checking happens.
<code>while (book <= 4);</code>	The condition, tested only after each run of the body. The semicolon at the end belongs to the do-while and must not be left out.
<code>book++;</code>	The update, inside the body. As in a while loop, forgetting it makes the loop endless.
<code>int extra = 12;</code>	Declared at the top with the other variable, and deliberately larger than 4, so the condition is false from the very beginning. A plain while loop would print nothing here; the do-while still prints once.

OUTPUT

```
Book 1 stamped at the Library counter
Book 2 stamped at the Library counter
Book 3 stamped at the Library counter
Book 4 stamped at the Library counter

Count is already 12, but this line still runs one time
```

Q4. Prayagraj Junction — Announcements Until the Train Leaves

REAL SITUATION

On the platform at Prayagraj Railway Junction.

The announcement for your train is repeated again and again over the loudspeaker. Nobody tells the announcer "say it a fixed number of times and stop" — it simply keeps repeating.

The only thing that ends it is the **train leaving**. In this program, the train leaves after the sixth announcement.

CONCEPT USED

An **infinite loop** written on purpose: while (1) means "repeat for ever", because 1 is always true.

Such a loop **must** contain a break, otherwise the program never ends. Here the break is the train leaving.

```
#include <stdio.h>

int main() {
    int call = 1;

    while (1) {                // 1 is always true: an endless loop

        printf("Announcement %d - train is on platform 1\n", call);
        call++;

        if (call > 6) {        // the train leaves
            printf("Train has left - announcements stopped");
            break;             // the only way out of while (1)
        }
    }

    return 0;
}
```

ON THE PLATFORM	→	IN THE CODE
The announcement repeats without any fixed count	→	while (1) { ... }
One more announcement is made	→	printf(...); call++;
The train leaves the platform	→	if (call > 6)
The loudspeaker goes quiet	→	break;

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>while (1)</code>	The condition is the constant 1, which is always true, so this loop can never end by itself. <code>for (; ;)</code> means exactly the same thing.
<code>call++;</code>	Written before the if, so that the count is already updated when the stopping test is made.
<code>if (call > 6)</code>	The stopping rule. In an endless loop this test is what replaces the missing loop condition.
<code>break;</code>	Ends the loop at once and jumps to the line after it. Remove this one line and the program prints for ever, and would have to be stopped with Ctrl + C.

OUTPUT

```
Announcement 1 - train is on platform 1
Announcement 2 - train is on platform 1
Announcement 3 - train is on platform 1
Announcement 4 - train is on platform 1
Announcement 5 - train is on platform 1
Announcement 6 - train is on platform 1
Train has left - announcements stopped
```


Q5. Room 5006 — Looking for an Empty Seat

REAL SITUATION

Theory class in Room 5006, and you are a little late.

There is a row of **10 seats**. You start from seat 1 and walk along the row, looking for a place to sit.

Seat 1, seat 2 and seat 3 are taken. **Seat 4 is empty** — and the moment you see it, you **stop looking** and sit down. There is no reason to walk on and check seats 5 to 10.

CONCEPT USED

break inside a **for** loop — it ends the loop early, from inside the body.

The loop is written for all 10 seats, because you do not know in advance which one will be free. The break stops it as soon as the search succeeds, and the update `seat++` does not run either.

```
#include <stdio.h>

int main() {
    int seat;
    int emptySeat = 4;                // seat 4 is the empty one today

    for (seat = 1; seat <= 10; seat++) {    // 10 seats in the row

        if (seat == emptySeat) {
            printf("Seat %d is empty - you sit down here\n", seat);
            break;                        // the search is over
        }

        printf("Seat %d is taken\n", seat);
    }

    printf("Seats checked before sitting = %d", seat);

    return 0;
}
```

IN THE ROW IN ROOM 5006	→	IN THE CODE
Ten seats to walk past	→	<code>for (seat = 1; seat <= 10; seat++)</code>
"Is this one empty?"	→	<code>if (seat == emptySeat)</code>
This one is taken, walk to the next	→	<code>printf("Seat %d is taken\n", seat);</code>
An empty seat is found, you stop looking	→	<code>break;</code>

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int emptySeat = 4;</code>	Stands for the first empty seat in the row. Change this one number to 7 and the search runs three seats further, up to seat 7, without any other change in the program.
<code>for (... seat <= 10 ...)</code>	The loop is written for the whole row of 10, because you cannot know in advance where the empty seat will be.
<code>break;</code>	Ends the for loop at once. Seats 5 to 10 are never checked, and control jumps to the printf written after the loop.
<code>printf("... = %d", seat);</code>	The break happened while seat was 4, and the value stays there, so the program counts the answer instead of having 4 typed into the message.

OUTPUT

```
Seat 1 is taken
Seat 2 is taken
Seat 3 is taken
Seat 4 is empty - you sit down here
Seats checked before sitting = 4
```

Q6. IIITA Main Gate — the Night Entry Register

REAL SITUATION

Night, 10:00 PM — the register at the IIITA Main Gate.

The guard goes down the list of roll numbers from **1 to 8** and notes the entry of each student returning to the hostel.

Roll number **3** and roll number **6** had already returned in the evening and signed the register then. The guard does not close the register for them — he simply **skips those two names** and calls the next roll number.

In the end, 6 fresh entries out of 8 are noted.

CONCEPT USED

continue inside a **for** loop — it skips only the rest of the current turn and goes on to the next one.

The loop itself is not stopped, which is exactly the difference from **break**.

The condition uses **||** (logical OR) from Assignment 2, because either of the two roll numbers should be skipped.

```
#include <stdio.h>

int main() {
    int roll;

    for (roll = 1; roll <= 8; roll++) {

        if (roll == 3 || roll == 6) {           // already signed in earlier
            printf("Roll number %d has already signed in - skipped\n", roll);
            continue;                          // skip the rest of this turn
        }

        printf("Entry noted for roll number %d\n", roll);
    }

    return 0;
}
```

AT THE MAIN GATE REGISTER	→	IN THE CODE
Going down the roll list, 1 to 8	→	for (roll = 1; roll <= 8; roll++)
Roll 3 or roll 6 has already signed in	→	if (roll == 3 roll == 6)
Skip that name, call the next one	→	continue;
A fresh entry is noted in the register	→	printf("Entry noted for roll number %d", roll);

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>if (roll == 3 roll == 6)</code>	True for either of the two roll numbers already signed in. <code>==</code> compares, and <code> </code> means "this one or that one".
<code>continue;</code>	Jumps straight to the update <code>roll++</code> and then to the condition. Every line written below it inside the body is skipped for this turn only.
<code>printf("Entry noted ...")</code>	Never reached for roll 3 and roll 6, because the <code>continue</code> jumped over it. It runs for the other six roll numbers.
<code>the loop keeps running</code>	This is the difference from <code>break</code> : after a <code>continue</code> , the loop comes back for the next roll number instead of ending.

OUTPUT

```
Entry noted for roll number 1
Entry noted for roll number 2
Roll number 3 has already signed in - skipped
Entry noted for roll number 4
Entry noted for roll number 5
Roll number 6 has already signed in - skipped
Entry noted for roll number 7
Entry noted for roll number 8
```

Q7. IIITA to Sangam — the Bus that Skips an Empty Stop

REAL SITUATION

Sunday morning — the bus going from IIITA to Sangam.

There are **6 stops** on the way. At every stop the driver slows down, picks up the students waiting there, and moves on.

At **stop number 4** there is nobody waiting. The driver does not end the trip there — he simply **does not halt**, and drives on to stop 5 and stop 6.

CONCEPT USED

continue inside a **while** loop.

In a while loop the update is inside the body, so it must be done **before** the continue. Here stop++ is written at the **top** of the body — if it were written at the bottom, the continue would jump over it, the bus would stay at stop 4 for ever and the program would never end.

```
#include <stdio.h>

int main() {
    int stop = 0;

    while (stop < 6) {
        stop++;                // update FIRST, so continue can never skip it

        if (stop == 4) {       // nobody is waiting here
            printf("Stop 4 - nobody waiting, the bus does not halt\n");
            continue;
        }

        printf("Stop %d - bus halts and students board\n", stop);
    }

    printf("Bus reaches Sangam");

    return 0;
}
```

ON THE WAY TO SANGAM	→	IN THE CODE
Six stops on the route	→	while (stop < 6)
The bus moves to the next stop	→	stop++; // at the TOP of the body
Nobody is waiting at stop 4	→	if (stop == 4) { ... continue; }
Students board at a normal stop	→	printf("Stop %d - bus halts ...", stop);

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int stop = 0;</code>	Started at 0, not 1, because the first thing inside the body is <code>stop++</code> . So the first stop reached is number 1.
<code>while (stop < 6)</code>	<code><</code> is used here, not <code><=</code> , because the counter is increased at the top of the body. The last stop reached is number 6.
<code>stop++;</code> (first line of the body)	The most important line of this program. Placing the update above the continue is what keeps the bus moving safely.
<code>continue;</code>	Jumps back to the condition <code>stop < 6</code> , skipping the boarding message for stop 4 only.

OUTPUT

```
Stop 1 - bus halts and students board
Stop 2 - bus halts and students board
Stop 3 - bus halts and students board
Stop 4 - nobody waiting, the bus does not halt
Stop 5 - bus halts and students board
Stop 6 - bus halts and students board
Bus reaches Sangam
```

Q8. Campus Trip Desk — Sangam, Civil Lines or the Junction

REAL SITUATION

Weekend — the trip desk near the IIITA Main Gate.

Three buses are leaving today, and each student books **one** of them by saying a single code:

- **S** → Sangam
- **C** → Civil Lines
- **J** → Prayagraj Railway Junction

Once your bus is booked, the clerk stops there. He does not go on and book the other two buses in your name as well.

CONCEPT USED

break inside a **switch** — the same keyword you used in loops, doing the same job: "stop here and come out".

Without any break, C keeps running the cases written below the matching one, so booking Sangam would print all three buses and the default line as well. That mistake is called **fall-through**.

```
#include <stdio.h>

int main() {
    char trip;

    printf("Enter trip code (S=Sangam, C=Civil Lines, J=Junction): ");
    scanf("%c", &trip);

    switch (trip) {
        case 'S':
            printf("Bus 1 leaves for Sangam");
            break;           // booked, come out of the switch

        case 'C':
            printf("Bus 2 leaves for Civil Lines");
            break;

        case 'J':
            printf("Bus 3 leaves for Prayagraj Railway Junction");
            break;

        default:              // no bus for this code
            printf("No trip is booked for this code");
    }

    return 0;
}
```

AT THE TRIP DESK	→	IN THE CODE
The one code you say at the desk	→	<code>char trip; scanf("%c", &trip);</code>
The clerk matches it against the three buses	→	<code>switch (trip) { case 'S': case 'C': case 'J': }</code>
Your bus is booked and the clerk stops	→	<code>break;</code>
The code is not on today's list	→	<code>default: printf("No trip is booked ...");</code>

Line by line

PART	WHAT IT DOES AND WHY IT IS NEEDED
<code>switch (trip)</code>	Takes the single character you typed and looks for the case that matches it exactly.
<code>case 'S': / 'C' / 'J'</code>	The three fixed trip codes. Each case must be a constant value written in single quotes.
<code>break;</code>	Ends the switch. Remove it from case 'S' and booking Sangam would also print the Civil Lines line, because control carries on into the next case until the next break is reached — the fall-through mistake.
<code>default:</code>	Runs when the typed code matches no case. It needs no break because it is the last block.

OUTPUT 1 — CODE S IS ENTERED

```
Enter trip code (S=Sangam, C=Civil Lines, J=Junction): S
Bus 1 leaves for Sangam
```

OUTPUT 2 — CODE J IS ENTERED

```
Enter trip code (S=Sangam, C=Civil Lines, J=Junction): J
Bus 3 leaves for Prayagraj Railway Junction
```

OUTPUT 3 — A CODE THAT IS NOT ON THE LIST

```
Enter trip code (S=Sangam, C=Civil Lines, J=Junction): X
No trip is booked for this code
```


Q9. IITA Pavilion — Numbering the Rows and Seats

REAL SITUATION

Match day — the IITA Pavilion.

Chairs are being arranged: **4 rows**, with **5 seats in every row**.

You number row 1 completely — seats 1 to 5 — and only then move to row 2 and number its five seats. Then row 3, then row 4.

Total chairs = $4 \times 5 = 20$.

CONCEPT USED

Nested for loops — a loop inside another loop.

The **outer** loop walks through the rows. For each single row, the **inner** loop runs completely, from seat 1 to seat 5.

The two loops use different counters, row and seat.

```
#include <stdio.h>

int main() {
    int row, seat;
    int chairs = 0;           // counts every chair numbered

    for (row = 1; row <= 4; row++) {           // OUTER loop: the rows

        for (seat = 1; seat <= 5; seat++) {    // INNER loop: seats of one row
            printf("Row %d Seat %d", row, seat);
            chairs++;                          // one more chair done
        }

        printf("\n");                        // one row finished
    }

    printf("Total chairs arranged = %d", chairs);

    return 0;
}
```

AT THE PAVILION	→	IN THE CODE
Moving from one row to the next	→	for (row = 1; row <= 4; row++)
Numbering the five seats of that row	→	for (seat = 1; seat <= 5; seat++)
Writing one chair number	→	printf("Row %d Seat %d", row, seat);
That row is finished, start a new line	→	printf("\n"); // in the OUTER body

Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
<code>int row, seat;</code>	Two separate counters. If both loops used the same name, the inner loop would destroy the outer loop's count.
<code>outer for</code>	Runs 4 times. Each single turn of it stands for one complete row.
<code>inner for</code>	Runs 5 times for every turn of the outer loop, and starts again from seat 1 each time. So its body runs $4 \times 5 = 20$ times in all.
<code>chairs++;</code>	Inside the inner body, so it runs once for every single chair. At the end it holds 20, which proves that the inner body ran $4 \times 5 = 20$ times.
<code>printf("\n");</code>	Written in the outer body, after the inner loop. That is what puts each row on its own line. Move it inside the inner loop and every seat would go on a separate line.

OUTPUT

```
Row 1 Seat 1  Row 1 Seat 2  Row 1 Seat 3  Row 1 Seat 4  Row 1 Seat 5
Row 2 Seat 1  Row 2 Seat 2  Row 2 Seat 3  Row 2 Seat 4  Row 2 Seat 5
Row 3 Seat 1  Row 3 Seat 2  Row 3 Seat 3  Row 3 Seat 4  Row 3 Seat 5
Row 4 Seat 1  Row 4 Seat 2  Row 4 Seat 3  Row 4 Seat 4  Row 4 Seat 5
Total chairs arranged = 20
```

Q10. Weekly Timetable — Lab in 5042 and Theory in 5006

REAL SITUATION

The class timetable for three days.

Each day has four periods, but two rules apply to every single day:

- **Period 2 is free** — there is no class, so it is **skipped** and the day continues
- **Period 4 is never held** — the day **ends** there and everyone leaves

So on each day, only period 1 and period 3 have a class: Lab in Room 5042 and Theory in Room 5006.

CONCEPT USED

Nested loops with **continue** and **break** working together.

The outer loop walks through the days. Inside it, **continue** skips one period and **break** ends the day.

Important: the **break** ends only the **inner** loop, so the next day still starts normally.

```
#include <stdio.h>

int main() {
    int day, period;

    for (day = 1; day <= 3; day++) {                // OUTER loop: the days
        printf("Day %d\n", day);

        for (period = 1; period <= 4; period++) {    // INNER loop: the periods

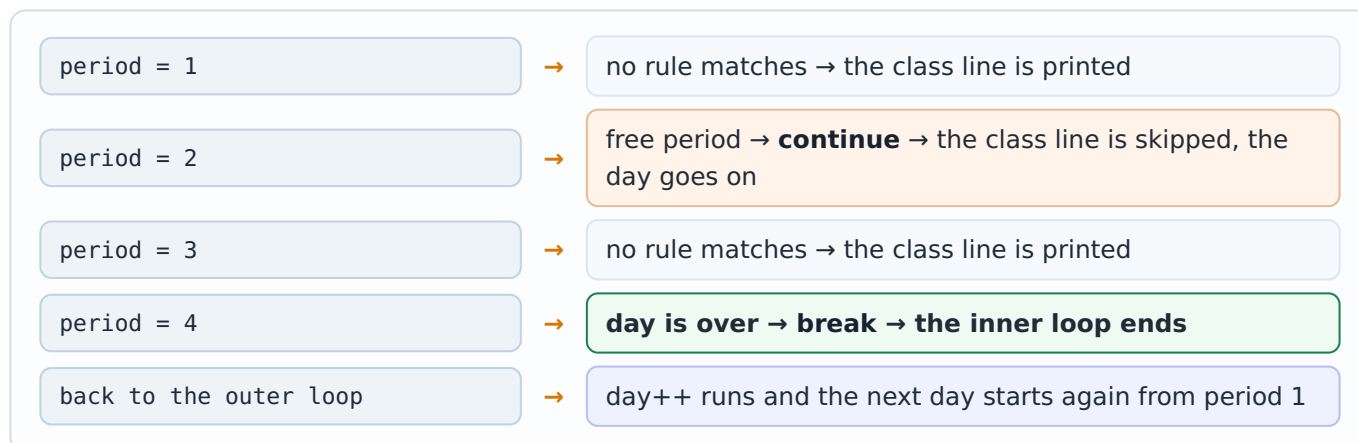
            if (period == 2) {                        // free period
                printf("    Period 2 - free, no class\n");
                continue;                             // skip, the day goes on
            }

            if (period == 4) {                        // day is over
                printf("    Day %d is over\n", day);
                break;                                // ends the INNER loop only
            }

            printf("    Period %d - Lab in 5042 / Theory in 5006\n", period);
        }
    }

    return 0;
}
```

Step by step — what happens inside one day



Line by line

LINE	WHAT IT DOES AND WHY IT IS WRITTEN
outer for (day ...)	Runs 3 times, once for each day. The inner loop starts again from period 1 on every one of those days.
continue; (period 2)	Skips only the rest of that one period and moves to period 3. The day is not ended.
break; (period 4)	Ends the inner loop only. The outer loop is untouched, which is why Day 2 and Day 3 still run.
indentation of the printf lines	The three spaces at the start of the inner messages are only for reading comfort, so the periods appear under their day.

OUTPUT

```
Day 1
  Period 1 - Lab in 5042 / Theory in 5006
  Period 2 - free, no class
  Period 3 - Lab in 5042 / Theory in 5006
  Day 1 is over
Day 2
  Period 1 - Lab in 5042 / Theory in 5006
  Period 2 - free, no class
  Period 3 - Lab in 5042 / Theory in 5006
  Day 2 is over
Day 3
  Period 1 - Lab in 5042 / Theory in 5006
  Period 2 - free, no class
  Period 3 - Lab in 5042 / Theory in 5006
  Day 3 is over
```

Quick Revision — Concept, Real IIITA Memory, Code Shape

CONCEPT	REMEMBER IT AS	CODE SHAPE
Loop and iteration	jogging at the SAC: the whole activity is the loop, one round is one iteration	repeat a block of work
Four parts of a loop	the Main Gate counter: start, check, work, update	init; condition; body; update
while	SAC track: the question is asked before every round, so it can run zero times	while (cond) { ... }
for	the road to Civil Lines: the exact number of kilometres is known before the work begins	for (i = 1; i <= n; i++)
do-while	the Library counter: the book is stamped first and the question is asked after, so it runs at least once	do { ... } while (cond);
Infinite loop	the Junction announcement that repeats until the train leaves; also what happens when the update is forgotten	while (1) { ... break; }
break	the search in Room 5006 ends the moment an empty seat is found; the trip is booked and the clerk stops	break; // leave the loop
continue	the broken chair left aside at the Auditorium; the roll number already signed in at the Main Gate; the bus stop where nobody is waiting	continue; // skip this turn
Nested loops	the Pavilion: for every row, all the seats of that row	for (...) { for (...) { } }

QUESTION	BREAK OR CONTINUE?
The work must stop completely and never come back	break
Only this one turn must be skipped, the work goes on	continue

— End of Assignment 3 —